

Java Streams

SECTION 1: STREAM CONCEPTS

1. What is a Stream in Java?

A Stream in Java is a sequence of elements that supports functional-style operations. It was introduced in Java 8.

A Stream is NOT a data structure. It does not store data. Instead, it processes data from a source such as a Collection, Array, or I/O channel.

Streams allow operations like filtering, mapping, sorting, and reducing in a clean and readable way.

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);  
numbers.stream()  
    .filter(n -> n % 2 == 0)  
    .forEach(System.out::println);
```

In this example, `stream()` creates a stream, `filter()` is an intermediate operation, and `forEach()` is a terminal operation.

2. Difference Between Collection and Stream

Collection stores data in memory. Stream processes data.

Collection can be traversed multiple times. Stream can be consumed only once.

Collection operations are eager. Stream operations are lazy (until terminal operation is called).

3. What is Stream Lifecycle?

Stream lifecycle consists of:

1. Source – Collection, array, or I/O.
2. Intermediate Operations – `filter()`, `map()`, `sorted()`, etc.
3. Terminal Operation – `collect()`, `forEach()`, `reduce()`, etc.

Without a terminal operation, the stream pipeline does not execute.

4. What is Lazy Evaluation?

Intermediate operations in Stream are lazy. They are executed only when a terminal operation is invoked.

```
List<String> list = Arrays.asList("A", "B", "C");  
  
list.stream()
```

```

        .filter(s -> {
            System.out.println("Filtering " + s);
            return true;
        }); // No output because no terminal operation

```

No output is printed because terminal operation is missing.

5. map() vs flatMap()

map() transforms each element individually.

flatMap() is used when each element produces multiple elements (like List of Lists).

```

List<List<String>> nested = Arrays.asList(
    Arrays.asList("A", "B"),
    Arrays.asList("C", "D")
);

nested.stream()
    .flatMap(List::stream)
    .forEach(System.out::println);

```

flatMap flattens nested collections into a single stream.

SECTION 2: PRACTICAL STREAM PROGRAMS

Employee Class Used in Examples

```

class Employee {
    private String name;
    private String department;
    private double salary;

    public Employee(String name, String department, double
salary) {
        this.name = name;
        this.department = department;
        this.salary = salary;
    }

    public String getName() { return name; }
    public String getDepartment() { return department; }
    public double getSalary() { return salary; }
}

```

1. Count Occurrence of Each Element

```

List<String> list =
Arrays.asList("A", "B", "A", "C", "B", "A", "D", "B");

```

```
Map<String, Long> frequency = list.stream()
    .collect(Collectors.groupingBy(
        s -> s,
        Collectors.counting()
    ));
```

```
System.out.println(frequency);
```

`groupingBy` groups elements and `counting()` counts occurrences.

2. Find Second Most Repeated Element (Basic Version)

```
String secondMost = list.stream()
    .collect(Collectors.groupingBy(s -> s,
    Collectors.counting()))
    .entrySet()
    .stream()
    .sorted(Map.Entry.<String,
    Long>comparingByValue().reversed())
    .skip(1)
    .map(Map.Entry::getKey)
    .findFirst()
    .orElse(null);
```

```
System.out.println(secondMost);
```

First we group and count. Then we sort by frequency in descending order, skip the first, and pick the second.

3. Find Duplicate Elements

```
Set<String> duplicates = list.stream()
    .collect(Collectors.groupingBy(s -> s,
    Collectors.counting()))
    .entrySet()
    .stream()
    .filter(e -> e.getValue() > 1)
    .map(Map.Entry::getKey)
    .collect(Collectors.toSet());
```

```
System.out.println(duplicates);
```

4. Group Employees by Department

```
List<Employee> employees = Arrays.asList(
    new Employee("John", "IT", 60000),
    new Employee("Alice", "HR", 50000),
    new Employee("Bob", "IT", 70000)
);
```

```
Map<String, List<Employee>> grouped = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDepartment));

System.out.println(grouped);
```

5. Find Highest Salary Employee

```
Employee highest = employees.stream()
    .max(Comparator.comparing(Employee::getSalary))
    .orElse(null);
```

```
System.out.println(highest.getName());
```

6. Partition Even and Odd Numbers

```
List<Integer> numbers = Arrays.asList(1,2,3,4,5,6);

Map<Boolean, List<Integer>> partitioned = numbers.stream()
    .collect(Collectors.partitioningBy(n -> n % 2 == 0));

System.out.println(partitioned);
```

partitioningBy divides elements into two groups based on condition (true/false).

SECTION 3: PRODUCT EDGE CONCEPTS

Why Stream Cannot Be Reused?

Once a terminal operation is executed, the stream is closed. Reusing it throws `IllegalStateException`.

```
Stream<String> stream = list.stream();
stream.forEach(System.out::println);
stream.forEach(System.out::println); // Error
```

Why Parallel Stream Can Be Risky?

Parallel streams use multiple threads.

If operations are not stateless or thread-safe, it may cause incorrect results.

Parallel stream should be used only when operations are independent and large data set exists.